

Amazon ML Challenge 2026 — Team A Playbook

Data Engineering, Text Normalization, Inverted Index Blocking & S3 Delivery

Amazon ML Challenge 2026 — Team A Playbook

Focus Area: Data Cleaning, Blocking, Normalization & Feature Pipeline

Operating Window: Friday 26th September (Full Day until 10:00 PM IST Hard Stop)

Target Milestone: Push clean `.parquet` feature sets to S3 before exam day (27th Sept).

1. Executive Summary & Team Mission

Team A has the most foundational responsibility in the entire hackathon:

If your blocking misses a true match, the ML model will never find it. If your feature computation crashes or leaks data, the model fails.

Your core mission on September 26:

- Never crash your machine:** Read the 3GB dataset efficiently without Out-Of-Memory (OOM) errors.
- Text Normalization:** Handle noisy business names, legal abbreviations, addresses, and Hindi (Devanagari) script.
- Smart Blocking:** Narrow down 26.4 million potential comparisons to ≤ 5\$ high-quality candidates per Source 1 entity.
- Fast Similarity Feature Extraction:** Compute rapid string similarity scores (`rapidfuzz`).
- Leak-Proof Train/Val Split:** Group by `source1_entity_id`.
- S3 Delivery by 10:00 PM:** Save final clean Parquet files so Team B can train models on September 27.

2. Team A Member Roles & Schedule

Member Role	Primary Focus	Key Responsibilities
Member 1 (Team Leader)	Architecture & Blocking	Inverted index blocking, country partitioning, S3 pipeline orchestration
Member 2 (Data Partner)	Normalization & Features	String normalization, Devanagari text handling, Rapidfuzz feature extraction

Timeline for Sept 26

- 09:00 AM - 10:30 AM:** AWS SageMaker instance upgrade to `ml.m5.xlarge` (16GB RAM) and S3 test.
- 10:30 AM - 01:00 PM: MILESTONE 1 (50k Sample):** Produce 50,000-entity sample Parquet and upload to `s3://<bucket>/processed/sample_50k/` for Team B.

- **02:00 PM - 06:00 PM:** Full-scale country-partitioned blocking on train & test sets.
- **06:00 PM - 09:30 PM:** Feature computation across all candidates in chunks.
- **09:30 PM - 10:00 PM: MILESTONE 2 (Full Data):** Save full Parquets to `s3://<bucket>/processed/full/`. Handoff complete!

3. Step-by-Step Implementation Guide

Step 0: Upgrade SageMaker Instance (Avoid OOM Crashes)

Your default `ml.t3.medium` has only 4GB RAM. Upgrading to `ml.m5.xlarge` gives you 16GB RAM and costs ~\$0.23/hr (using your \$200 credits):

1. In SageMaker Console \$
 `rightarrow`\$ Notebook Instances \$
 `rightarrow`\$ Select notebook \$
 `rightarrow`\$ **Stop**.
2. Click **Actions** \$
 `rightarrow`\$ **Edit** \$
 `rightarrow`\$ Change instance type to `ml.m5.xlarge` \$
 `rightarrow`\$ **Save**.
3. Click **Start**. You now have 16GB RAM!

Install dependencies in your terminal:

```
pip install rapidfuzz lightgbm pyarrow s3fs boto3 tqdm
```

Step 1: Text Normalization (Multilingual Safe)

India records include Hindi Devanagari text (e.g., राम मार्केटिंग), and test data contains French names. Use Unicode NFKD normalization:

```
import re
import unicodedata

def clean_text(text):
    if not isinstance(text, str):
        return ""
    # Normalize unicode accents and scripts cleanly
    text = unicodedata.normalize('NFKD', text).lower().strip()

    # Standardize common business suffixes
    text = re.sub(r'\b(pvt|private)\b', 'pvt', text)
    text = re.sub(r'\b(ltd|limited)\b', 'ltd', text)
    text = re.sub(r'\b(inc|incorporated)\b', 'inc', text)
    text = re.sub(r'\b(corp|corporation)\b', 'corp', text)
    text = re.sub(r'\b(llc|llp)\b', 'llc', text)
    text = re.sub(r'\b&\b', 'and', text)

    # Address abbreviations
    text = re.sub(r'\b(rd|road)\b', 'road', text)
    text = re.sub(r'\b(st|street)\b', 'street', text)
    text = re.sub(r'\b(ave|avenue)\b', 'avenue', text)

    # Remove unwanted punctuation but keep alphanumeric and Hindi characters
    text = re.sub(r'[\^\\w\s]', ' ', text)
    return re.sub(r'\s+', ' ', text).strip()
```

Step 2: Country-Partitioned Inverted Index Blocking

Never compare cross-country. Build an inverted index on significant words (length ≥ 3):

```
from collections import defaultdict
import pandas as pd

def build_candidates_for_country(s1_df, s23_df, max_candidates=5):
    # 1. Map tokens -> list of S2/S3 row indices
    index = defaultdict(list)
    for idx, name in enumerate(s23_df['clean_name']):
        for token in set(name.split()):
            if len(token) >= 3:
                index[token].append(idx)

    # 2. Match S1 records against index
    candidate_pairs = []
    for _, s1 in s1_df.iterrows():
        s1_id = s1['entity_id']
        tokens = [t for t in set(s1['clean_name'].split()) if len(t) >= 3]

        counts = defaultdict(int)
        for t in tokens:
            for s23_idx in index.get(t, []):
                counts[s23_idx] += 1

        if not counts:
            continue

        top_matches = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:max_candidates]
        for s23_idx, _ in top_matches:
            candidate_pairs.append({
                'source1_entity_id': s1_id,
                'candidate_entity_id': s23_df.iloc[s23_idx]['entity_id'],
                's1_name': s1['clean_name'],
                's23_name': s23_df.iloc[s23_idx]['clean_name'],
                's1_addr': s1['clean_addr'],
                's23_addr': s23_df.iloc[s23_idx]['clean_addr'],
                'country': s1['country']
            })
    return pd.DataFrame(candidate_pairs)
```

Step 3: Feature Extraction with Rapidfuzz

Compute similarity features that the gradient boosting model will learn from:

```
from rapidfuzz import fuzz

def compute_features(df):
    features = pd.DataFrame()
    features['source1_entity_id'] = df['source1_entity_id']
    features['candidate_entity_id'] = df['candidate_entity_id']

    # Rapid name similarity metrics (0 to 1)
    features['name_ratio'] = [fuzz.ratio(n1, n2)/100.0 for n1, n2 in zip(df['s1_name'], df['s23_name'])]
    features['name_partial'] = [fuzz.partial_ratio(n1, n2)/100.0 for n1, n2 in zip(df['s1_name'], df['s23_name'])]
    features['name_token_sort'] = [fuzz.token_sort_ratio(n1, n2)/100.0 for n1, n2 in zip(df['s1_name'], df['s23_name'])]
    features['name_token_set'] = [fuzz.token_set_ratio(n1, n2)/100.0 for n1, n2 in zip(df['s1_name'], df['s23_name'])]

    # Address similarity metrics (0 to 1)
    features['addr_ratio'] = [fuzz.ratio(a1, a2)/100.0 for a1, a2 in zip(df['s1_addr'], df['s23_addr'])]
    features['addr_partial'] = [fuzz.partial_ratio(a1, a2)/100.0 for a1, a2 in zip(df['s1_addr'], df['s23_addr'])]
```

```

features['addr_token_set'] = [fuzz.token_set_ratio(a1, a2)/100.0 for a1, a2 in zip(df['s1_addr'], df['s23_addr'])]

# Length differences
features['name_len_diff'] = [abs(len(n1) - len(n2)) for n1, n2 in zip(df['s1_name'], df['s23_name'])]
features['addr_len_diff'] = [abs(len(a1) - len(a2)) for a1, a2 in zip(df['s1_addr'], df['s23_addr'])]
return features

```

Step 4: Leak-Proof Train/Val Split & S3 Export

Group strictly by `source1_entity_id` so the same business never appears in both train and validation:

```

from sklearn.model_selection import GroupShuffleSplit

# Attach binary label from ground truth
# (label = 1 if (s1, candidate) in ground_truth else 0)
gss = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
train_idx, val_idx = next(gss.split(features_df, groups=features_df['source1_entity_id']))

train_data = features_df.iloc[train_idx]
val_data = features_df.iloc[val_idx]

# Save to S3 in Parquet format
train_data.to_parquet("s3://<your-bucket>/processed/full/train_candidates.parquet", index=False)
val_data.to_parquet("s3://<your-bucket>/processed/full/val_candidates.parquet", index=False)
print("Team A mission complete! Parquets saved to S3.")

```

4. Team A Golden Rules & Anti-Mistakes

- **DO NOT** read TSVs without `sep="\t"`.
- **DO NOT** perform random row-based train/val splits. Group by `source1_entity_id`.
- **DO NOT** compare entities across different countries.
- **DO NOT** wait until night to give Team B data. Always deliver the 50,000 sample by 1:00 PM!